

Project Executable Files

Date	28 June 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	3 Marks

Project Executable Files

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	No
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Credit Card Approval Prediction/

```

— app.py
— train.py
— random_forest_model.pkl
— clean_dataset.csv
— requirements.txt
— README.md
— .gitignore

— templates/
  — index.html
  — result.html

— static/
  — style.css
  — images/

— screenshots/
  — Home_Page.png
  — Approved_Result.png
  — Rejected_Result.png

— test_cases/
  — Test_Results.pdf

— documentation/
  — Project_Report.pdf
  — Demo_Video_Link.txt
  
```

Step 3: Deployment / Access Details

Field	Details
Deployed URL	N/A
Login Credentials (Demo)	243ta04037@recw.ac.in 243TA04037
Platform / Hosting Provider	Local System (Flask)
Repository Link	https://github.com/243ta04037-induja/Credit-Card-Approval-Prediction/new/main
Demo Video Link	https://drive.google.com/file/d/1rd8mP0BBYL9UmlqOc_1YTfkfZ32oMFhl/view?usp=sharing

Step 4: Run Instructions

1. Install **Python 3.x** on your system.
2. Open the **Credit Card Approval Prediction** project folder in **Visual Studio Code**.
3. Install the required Python libraries using:

```
pip install -r requirements.txt
```

4. Run the Flask application using:

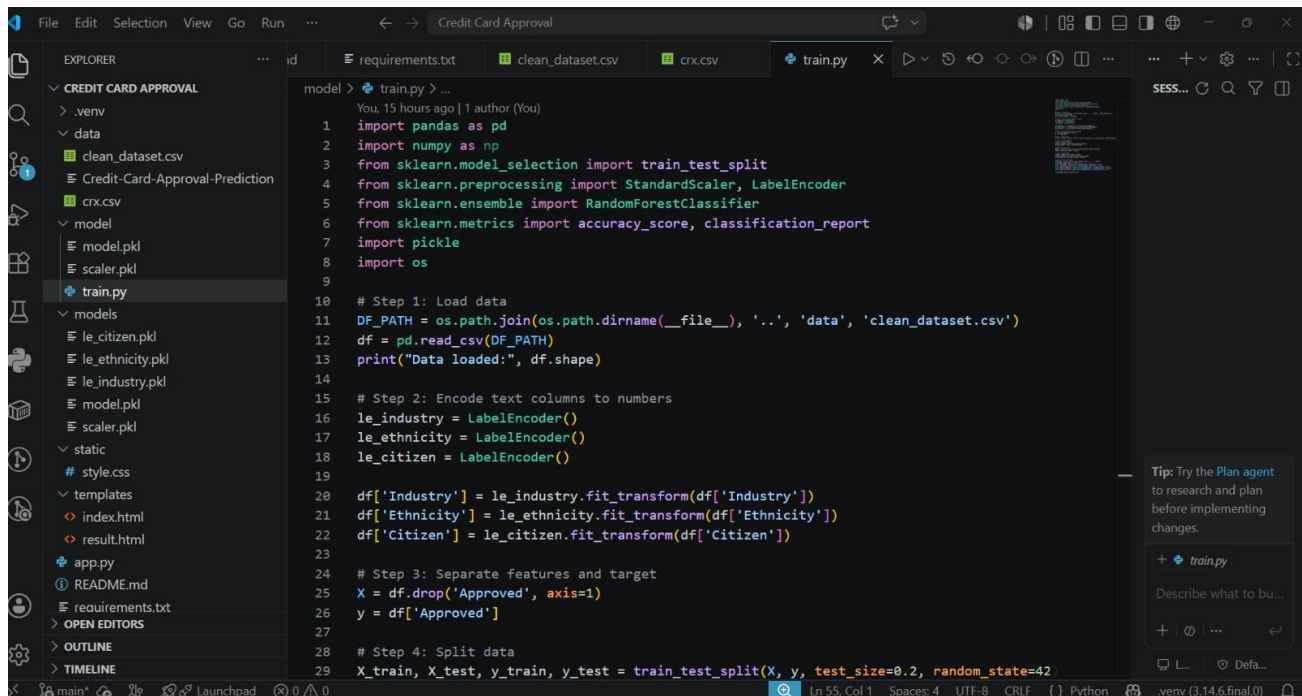
```
python app.py
```

5. Open the URL displayed in the terminal (usually <http://127.0.0.1:5000>) in your web browser.
6. Enter the required applicant details, such as age, income, employment status, credit score, and other information.
7. Click the **"Predict Approval"** button.
8. The system will process the input using the trained **Random Forest** machine learning model and display the prediction as **Approved** or **Rejected**, along with the confidence score.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy depends on the quality and size of the training dataset.	Use a larger, more diverse, and better-quality dataset to improve accuracy.
2	The application currently runs only on the local system using the Flask development server.	Deploy the application on Render, PythonAnywhere, or another cloud platform.
3	Applicant details are not stored in a database.	Integrate a database such as MySQL or SQLite in future versions.
4	The system does not provide user login or authentication.	Add user authentication and authorization in future updates.
5	The model currently uses only the Random Forest algorithm.	Compare and optimize performance using algorithms such as XGBoost, Gradient Boosting, or Neural Networks.

Code Implementation Screenshots



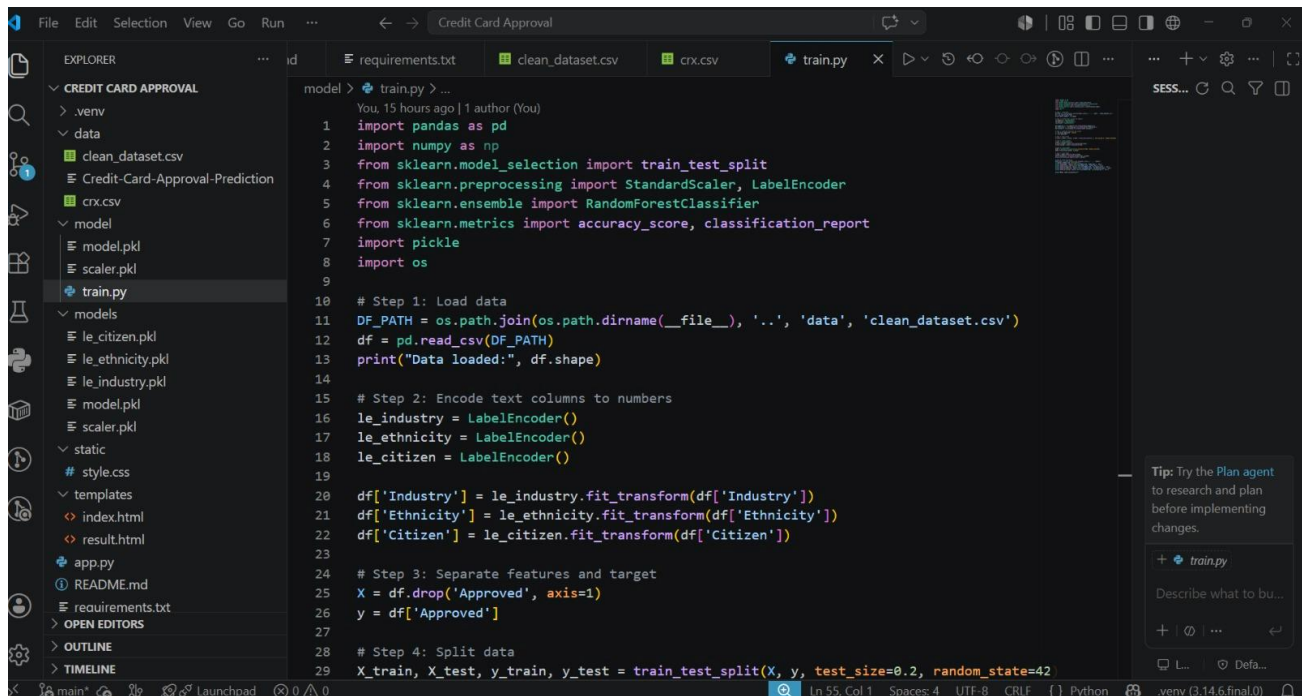
The screenshot shows a VS Code editor window with the 'train.py' file open. The file contains the following code:

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import StandardScaler, LabelEncoder
5  from sklearn.ensemble import RandomForestClassifier
6  from sklearn.metrics import accuracy_score, classification_report
7  import pickle
8  import os
9
10 # Step 1: Load data
11 DF_PATH = os.path.join(os.path.dirname(__file__), '..', 'data', 'clean_dataset.csv')
12 df = pd.read_csv(DF_PATH)
13 print("Data loaded:", df.shape)
14
15 # Step 2: Encode text columns to numbers
16 le_industry = LabelEncoder()
17 le_ethnicity = LabelEncoder()
18 le_citizen = LabelEncoder()
19
20 df['Industry'] = le_industry.fit_transform(df['Industry'])
21 df['Ethnicity'] = le_ethnicity.fit_transform(df['Ethnicity'])
22 df['Citizen'] = le_citizen.fit_transform(df['Citizen'])
23
24 # Step 3: Separate features and target
25 X = df.drop('Approved', axis=1)
26 y = df['Approved']
27
28 # Step 4: Split data
29 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42

```

The Explorer panel on the left shows the project structure, including files like 'requirements.txt', 'clean_dataset.csv', 'crx.csv', 'train.py', 'model.pkl', 'scaler.pkl', 'le_citizen.pkl', 'le_ethnicity.pkl', 'le_industry.pkl', 'static', 'style.css', 'templates', 'index.html', 'result.html', 'app.py', 'README.md', 'requirements.txt', 'OPEN EDITORS', 'OUTLINE', and 'TIMELINE'.



The screenshot shows a VS Code editor window with the 'train.py' file open. The file contains the following code:

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import StandardScaler, LabelEncoder
5  from sklearn.ensemble import RandomForestClassifier
6  from sklearn.metrics import accuracy_score, classification_report
7  import pickle
8  import os
9
10 # Step 1: Load data
11 DF_PATH = os.path.join(os.path.dirname(__file__), '..', 'data', 'clean_dataset.csv')
12 df = pd.read_csv(DF_PATH)
13 print("Data loaded:", df.shape)
14
15 # Step 2: Encode text columns to numbers
16 le_industry = LabelEncoder()
17 le_ethnicity = LabelEncoder()
18 le_citizen = LabelEncoder()
19
20 df['Industry'] = le_industry.fit_transform(df['Industry'])
21 df['Ethnicity'] = le_ethnicity.fit_transform(df['Ethnicity'])
22 df['Citizen'] = le_citizen.fit_transform(df['Citizen'])
23
24 # Step 3: Separate features and target
25 X = df.drop('Approved', axis=1)
26 y = df['Approved']
27
28 # Step 4: Split data
29 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42

```

The Explorer panel on the left shows the project structure, including files like 'requirements.txt', 'clean_dataset.csv', 'crx.csv', 'train.py', 'model.pkl', 'scaler.pkl', 'le_citizen.pkl', 'le_ethnicity.pkl', 'le_industry.pkl', 'static', 'style.css', 'templates', 'index.html', 'result.html', 'app.py', 'README.md', 'requirements.txt', 'OPEN EDITORS', 'OUTLINE', and 'TIMELINE'.

File Edit Selection View Go Run ... Credit Card Approval

EXPLORER

- CREDIT CARD APPROVAL
 - .venv
 - data
 - clean_dataset.csv
 - Credit-Card-Approval-Prediction
 - crx.csv
 - model
 - model.pkl
 - scaler.pkl
 - train.py
 - models
 - le_citizen.pkl
 - le_ethnicity.pkl
 - le_industry.pkl
 - model.pkl
 - scaler.pkl
 - static
 - style.css
 - templates
 - index.html
 - result.html
 - app.py
 - README.md
 - requirements.txt
 - OPEN EDITORS
 - OUTLINE
 - TIMELINE

```

model > train.py > ...
40 # Step 7: Test model
41 y_pred = model.predict(X_test_scaled)
42 print("Accuracy:", accuracy_score(y_test, y_pred))
43 print(classification_report(y_test, y_pred))
44
45 # Step 8: Save everything
46 MODEL_DIR = os.path.join(os.path.dirname(__file__), '..', 'models')
47 os.makedirs(MODEL_DIR, exist_ok=True)
48 pickle.dump(model, open(os.path.join(MODEL_DIR, 'model.pkl'), 'wb'))
49 pickle.dump(scaler, open(os.path.join(MODEL_DIR, 'scaler.pkl'), 'wb'))
50 pickle.dump(le_industry, open(os.path.join(MODEL_DIR, 'le_industry.pkl'), 'wb'))
51 pickle.dump(le_ethnicity, open(os.path.join(MODEL_DIR, 'le_ethnicity.pkl'), 'wb'))
52 pickle.dump(le_citizen, open(os.path.join(MODEL_DIR, 'le_citizen.pkl'), 'wb'))
53
54 print("Model saved successfully!")
55

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```

(.venv) PS D:\Credit Card Approval> & "d:\Credit Card Approval\.venv\Scripts\python.exe" "d:\Credit Card Ap
proval\model\train.py"

```

	precision	recall	f1-score	support
0	0.82	0.90	0.86	68
1	0.89	0.81	0.85	70
accuracy			0.86	138
macro avg	0.86	0.86	0.85	138
weighted avg	0.86	0.86	0.85	138

Model saved successfully!

```

(.venv) PS D:\Credit Card Approval>

```

Tip: Try the Plan agent to research and plan before implementing changes.

+ train.py

Describe what to bu...

+ @ ...

Ln 55, Col 1 Spaces: 4 UTF-8 CRLF Python .venv (3.14.6.final.0)

File Edit Selection View Go Run ... Credit Card Approval

EXPLORER

- CREDIT CARD APPROVAL
 - .venv
 - data
 - clean_dataset.csv
 - Credit-Card-Approval-Prediction
 - crx.csv
 - model
 - model.pkl
 - scaler.pkl
 - train.py
 - models
 - le_citizen.pkl
 - le_ethnicity.pkl
 - le_industry.pkl
 - model.pkl
 - scaler.pkl
 - static
 - style.css
 - templates
 - index.html
 - result.html
 - app.py
 - README.md
 - requirements.txt
 - OPEN EDITORS
 - OUTLINE
 - TIMELINE

```

app.py > predict
You, 15 hours ago | 1 author (You)
1 from flask import Flask, render_template, request
2 import pandas as pd
3 import pickle
4 import numpy as np
5
6 app = Flask(__name__)
7
8 # Load saved model and tools
9 model = pickle.load(open('models/model.pkl', 'rb'))
10 scaler = pickle.load(open('models/scaler.pkl', 'rb'))
11 le_industry = pickle.load(open('models/le_industry.pkl', 'rb'))
12 le_ethnicity = pickle.load(open('models/le_ethnicity.pkl', 'rb'))
13 le_citizen = pickle.load(open('models/le_citizen.pkl', 'rb'))
14
15 @app.route('/')
16 def home():
17     return render_template('index.html')
18
19 @app.route('/predict', methods=['POST'])
20 def predict():
21     # Get form data
22     gender = int(request.form['gender'])
23     age = float(request.form['age'])
24     debt = float(request.form['debt'])
25     married = int(request.form['married'])
26     bank_customer = int(request.form['bank_customer'])
27     industry = request.form['industry']
28     ethnicity = request.form['ethnicity']
29     years_employed = float(request.form['years_employed'])

```

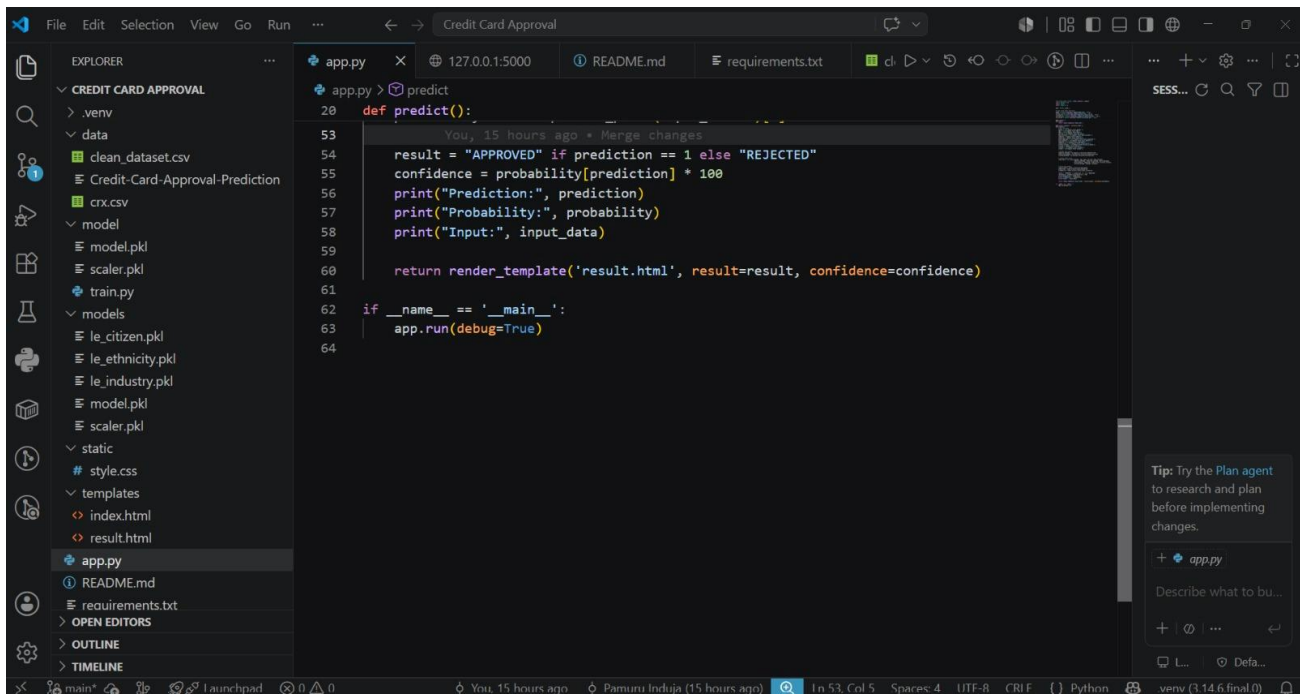
Tip: Try the Plan agent to research and plan before implementing changes.

+ app.py

Describe what to bu...

+ @ ...

Ln 53, Col 5 Spaces: 4 UTF-8 CRLF Python .venv (3.14.6.final.0)



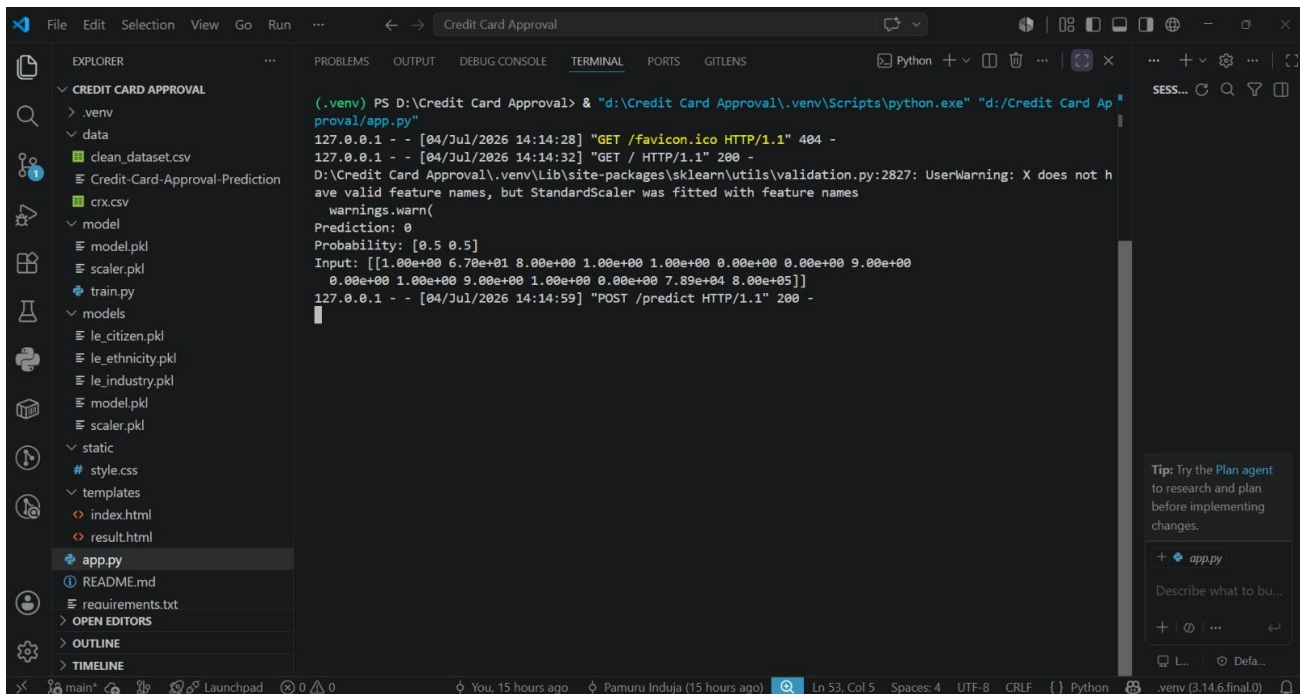
The image shows a VS Code editor window with the file explorer on the left displaying the project structure for 'CREDIT CARD APPROVAL'. The main editor area shows the 'app.py' file with the following code:

```

20 def predict():
21     You, 15 hours ago • Merge changes
22     result = "APPROVED" if prediction == 1 else "REJECTED"
23     confidence = probability[prediction] * 100
24     print("Prediction:", prediction)
25     print("Probability:", probability)
26     print("Input:", input_data)
27
28     return render_template('result.html', result=result, confidence=confidence)
29
30 if __name__ == '__main__':
31     app.run(debug=True)
32

```

The status bar at the bottom indicates the file is at line 53, column 5, with 4 spaces, UTF-8 encoding, CRLF line endings, and Python syntax highlighting. The bottom right corner shows the virtual environment path: '.venv (3.14.6.final.0)'.



The image shows the same VS Code editor window with the terminal panel open at the bottom. The terminal displays the output of running the application:

```

(.venv) PS D:\Credit Card Approval> & "d:\Credit Card Approval\.venv\Scripts\python.exe" "d:\Credit Card Ap
proval/app.py"
127.0.0.1 - - [04/Jul/2026 14:14:28] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [04/Jul/2026 14:14:32] "GET / HTTP/1.1" 200 -
D:\Credit Card Approval\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not h
ave valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(
Prediction: 0
Probability: [0.5 0.5]
Input: [[1.00e+00 6.70e+01 8.00e+00 1.00e+00 1.00e+00 0.00e+00 0.00e+00 9.00e+00
0.00e+00 1.00e+00 9.00e+00 1.00e+00 0.00e+00 7.89e+04 8.00e+05]]
127.0.0.1 - - [04/Jul/2026 14:14:59] "POST /predict HTTP/1.1" 200 -

```

The status bar at the bottom indicates the file is at line 53, column 5, with 4 spaces, UTF-8 encoding, CRLF line endings, and Python syntax highlighting. The bottom right corner shows the virtual environment path: '.venv (3.14.6.final.0)'.